- Decision Trees
  - Ensemble Techniques
    - Type
      - Bagging
      - Random Forest → a lot of trees (DT)
  - Bias - Variance
  - Sb - learn
  - Space / time complexity

Decision Trees

- Supervised (need Target)
- Classification (Binary, categorical, multi-class)
- "Tree-like" ✓

Classification

↳ node (Predicted node)

↳ node complexity → Impurity → IG = (Impurity (Parent) − Impurity (Children))

$p_1 = \frac{c_1}{n}$, $p_2 = \frac{c_2}{n}$, $p(x) = \frac{n_x}{n}$

$Gini = 1 - \sum_{i=1}^{c} (p_i)^2$

$c = 2$

- complexity
- Root Count
- impurity

Instant impurity (=0)

$d = 5$

Parent feat → Impurity → (Node)

$ch_1 = \frac{2n}{n+2n}$, $ch_2 = \frac{n}{n+2n}$

Absolute area $|g - g_1|$

Ensemble Techniques

↳ group of things

↳ multiple models

$\underbrace{M_1, M_2, M_3, \ldots\ldots, M_k}_{\text{combine}} \rightarrow$ Base learners

$n \rightarrow$ Powerful model

1. Bagging (Random Forest)
2. Boosting
3. Stacking (Kaggle) (Google DS competition)
4. Cascading

Bagging → (Bootstrap Aggregating)

↳ average (mean), median, mode

"sampling with replacement"

(D) → sample size while bootstrapping.

n (total) data points (Train) → $m < n$ (D) → m (D)

Node 1 Variance → ↑↑ → overfit

(Very high variance)

Bagging → sampling (replacement) → reduce variance

we will use BAGGING to tackle base model variance

↳ high variance + low bias

DT (will have good depth)

$M = \{M_1, M_2, \ldots, M_k\}$

low variance, low bias because bagging ← high variance, low bias

Sampling / Randomization → Reduce variance

Random Forest

↳ group of trees (DT)

↳ Bootstrapping (reduce variance)

RF ⇒ DT + Bagging + Col sampling (Base) → Bag

$n' << n$

$d' << d$

$n = 10000$, $n' = 10000$

$d = 100$, $d' = 20, 35$

Q1 → will RF alway outperform DT?

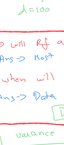
Ans → Most of the time, Yes

Q2 when will RF ∈ DT, Ref the same?

Ans → Data is simple

DT

why variance reduces? (sampling)



① depth of the learners

② RF → help reduce the variance

k (n_estimators)

Time & space complexity

Train 1 DT → (n log n · d)

Prediction time → O (Depth)

RF → Train Time

↳ Prediction time

(Parallel)

(Multi-processing)

Space complexity = DT

= O (nodes)

RF = O (nodes × k)